

出餐 出餐

详细设计说明书

小组成员：待确认（项目仓库未提供成员名单）

2026 年 9 月 10 日

版本：V3.0

文档范围

本文档描述仓库当前版本中可由源码、资源文件和 Git 历史验证的详细设计。项目采用 Java 17、JavaFX 17.0.10、FXML、Canvas 和 Maven；当前已形成页面切换、菜单音乐、60 秒倒计时及玩家移动的最小运行闭环。

实现状态分为已实现、部分实现、未实现和待确认。README 或需求分析中的目标只有在源码存在可执行落点时才记为已实现；只有类型、接口或空方法时记为部分实现或未实现。

待确认：小组成员名单、正式验收口径、关卡数量和菜谱内容未出现在当前工作树中。AI 使用信息依据 Git 历史中的 AI 记录.md；没有逐方法标注的内容保留为不确定。

一 状态机图与状态转移表

1.1 主状态机

GameState 定义 READY、RUNNING、PAUSED 和 FINISHED 四个状态，但当前 GameController 没有 currentState 字段，也没有统一的 setState 方法。实际生命周期由 finished 布尔值、GameTimer.running 和页面回调共同表达。下表把枚举中的设计意图与当前可执行路径分开列出。

当前状态	触发事件	前置条件	下一状态	说明
READY	GameView.initialize 调用 startGame	FXML 已加载， gameSeconds=60	RUNNING	实际代码启动计时；未写入 GameState 字段
RUNNING	AnimationTimer 每帧回调	finished=false	RUNNING	更新 Player 和 GameTimer
RUNNING	倒计时归零	secondsCount 递减至 0	FINISHED	GameTimer 调用 finishGame，随后 切换结果页
RUNNING	GameView.dispose 调用 stopGame	游戏页面被替换或应用退出	FINISHED	停止循环和计时；不触发结果回调
RUNNING	暂停操作	待确认：暂停入口与按键	PAUSED	未实现
PAUSED	恢复操作	待确认：恢复入口	RUNNING	未实现
FINISHED	结果页 restartGame	结果页控制器已配置	READY 后进入 RUNNING	回到 game.fxml 后新建 GameController；result.fxml 当前 为空，按钮未落地

1.2 状态枚举定义

```
public enum GameState {  
    READY,  
    RUNNING,  
    PAUSED,  
    FINISHED  
}
```

该枚举存在于 `com.orderup.model.GameState`，但当前无其他源码引用。`PAUSED` 因此仅是预留设计，不能视为已实现功能。

1.3 状态切换方法

当前没有模板示例中的 `setState` 方法。可执行切换分别落在 `GameController.startGame`、`finishGame` 和 `stopGame` 中。`finishGame` 通过 `finished` 标志保证回调只执行一次。

```
public void startGame() {  
    finished = false;  
    gameTimer.startCountDown(gameSeconds);  
}  
  
public void finishGame() {  
    if (finished) return;  
    finished = true;  
    gameTimer.stop();  
    player.clearMovement();  
    onGameFinished.run();  
}
```

1.4 主状态机图

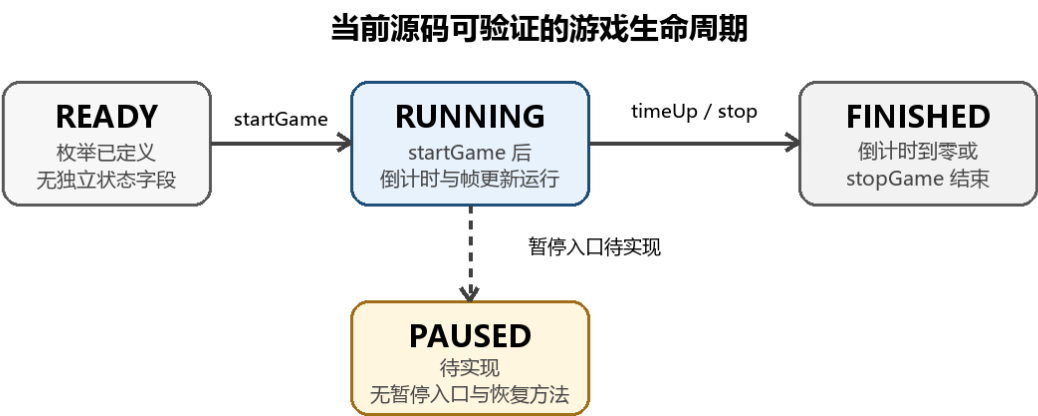


图 1 当前源码可验证的主状态流转

1.5 状态机自查

检查项	结果	说明
无死状态	部分满足	FINISHED 可经重新加载游戏页重新开始；PAUSED 无进入和退出实现
无不可达状态	不满足	GameState.PAUSED 在当前源码中不可达；GameState 整体未接入控制器
覆盖 P0 全部流程	不满足	当前仅覆盖启动、计时、移动和结束；订单、加工、装盘、出餐、评分、暂停等未形成闭环

1.6 子状态机

玩家移动输入可视为 RUNNING 下的概念子状态。源码没有为它定义枚举，而是使用 EnumSet<Direction> pressedDirections：集合为空时玩家静止，集合非空时按各方向合成位移。

当前子状态	触发事件	前置条件	下一子状态	说明
IDLE	按下 W A S D 任一键	Canvas 获得焦点	MOVING	GameView 转换为 Direction 并调用 controller.press
MOVING	继续按住或新增方向键	游戏未结束	MOVING	允许多方向同时存在；对角线速度未归一化
MOVING	释放部分方向键	仍有其他方向键按下	MOVING	移除对应 Direction

当前子状态	触发事件	前置条件	下一子状态	说明
MOVING	释放最后一个方向键	方向集合变为空	IDLE	位置停止更新
MOVING	Canvas 失焦或游戏结束	监听到失焦或调用 finishGame	IDLE	clearMovement 清空输入

1.7 子状态机图

RUNNING 状态下的玩家移动输入子状态

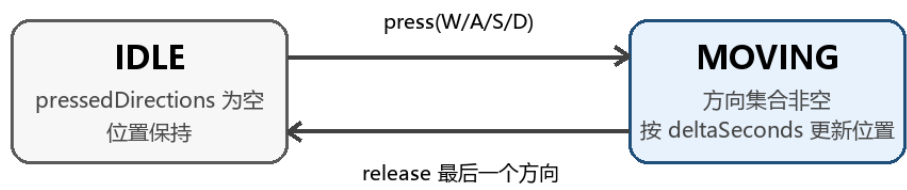


图 2 玩家移动输入子状态

二 方法级设计

以下方法签名和行为均来自当前源码。对空值、负速度等未校验情形只描述现状，不补写未存在的约束。

2.1 Launcher

职责：JavaFX 应用入口、窗口配置、FXML 页面切换和菜单音乐生命周期。

方法名	方法签名	参数说明	返回值	前置条件	后置条件
start	void start(Stage stage)	stage: 主窗口	无	JavaFX 调用 Application.start	设置窗口标题、最小尺寸并显示开始页
showScene	void showScene(String fxmlPath)	fxmlPath: 类路径资源	无	资源存在且 FXML 可加载	释放旧 GameView，切换根节点并配置控制器
configureController	void configureController(Object controller)	controller: FXML 控制器	无	FXML 已加载	为开始页、结果页或游戏页注入页面切换回调
playMenuMusic	void playMenuMusic()	无	无	startmenu.mp3 可选存在	创建或复用 MediaPlayer 并循环播放，音量 0.35

方法名	方法签名	参数说明	返回值	前置条件	后置条件
stop	void stop()	无	无	应用退出	释放 GameView 与菜单 MediaPlayer

2.2 GameView

职责：FXML 显示层、键盘输入、逐帧循环、画布绘制与倒计时展示。

方法名	方法签名	参数说明	返回值	前置条件	后置条件
initialize	void initialize()	无	无	FXML 字段已注入	创建 GameController，启动 60 秒倒计时、输入与游戏循环
startGameLoop	void startGameLoop()	无	无	controller 与 Canvas 已初始化	每帧计算不超过 0.05 秒的 deltaSeconds 并更新、绘制
renderFrame	void renderFrame(GraphicsContext graphics)	graphics：画布绘图上下文	无	graphics 非空	清空画布，绘制玩家并刷新倒计时显示
renderTime	void renderTime(int totalSeconds)	totalSeconds：剩余秒数	无	秒数由 GameTimer 提供	显示 mm:ss；剩余 10 秒及以下变为红色
dispose	void dispose()	无	无	页面将被替换或应用退出	停止 AnimationTimer 和 GameController

2.3 GameController

职责：协调玩家、计时器和游戏结束条件。

方法名	方法签名	参数说明	返回值	前置条件	后置条件
构造方法	GameController(double worldWidth, double worldHeight, int gameSeconds, Runnable onGameFinished)	世界尺寸、局时长、结束回调	新实例	参数由 GameView 提供	玩家初始位置为 200,200，并创建 GameTimer
startGame	void startGame()	无	无	gameSeconds 大于 0	finished=false，启动倒计时
update	void update(double deltaSeconds)	deltaSeconds：本帧秒数	无	游戏未结束； deltaSeconds 非负	更新玩家位置和倒计时

方法名	方法签名	参数说明	返回值	前置条件	后置条件
finishGame	void finishGame()	无	无	尚未结束	幂等地停止计时、清空输入并调用结束回调
stopGame	void stopGame()	无	无	页面销毁	直接标记结束并停止计时、清空输入

2.4 Player

职责：保存移动输入、位置和速度并执行边界限制。

方法名	方法签名	参数说明	返回值	前置条件	后置条件
press	void press(Direction direction)	direction：方向枚举	无	direction 非空	将方向加入 EnumSet
release	void release(Direction direction)	direction：方向枚举	无	无	从 EnumSet 移除方向
clearMovement	void clearMovement()	无	无	无	清空全部已按下方向
update	void update(double deltaSeconds, double worldWidth, double worldHeight)	帧时间与世界尺寸	无	世界尺寸不小于玩家尺寸	按速度更新位置，并限制在画布边界内
setSpeed	void setSpeed(double speed)	speed：像素每秒	无	源码未校验非负值	更新移动速度；负值处理规则待确认

2.5 GameTimer

职责：与 JavaFX 控件解耦的帧驱动倒计时。

方法名	方法签名	参数说明	返回值	前置条件	后置条件
构造方法	GameTimer(Runnable onTimeUp)	onTimeUp：超时回调	新实例	回调应非空；源码未校验	保存回调，计时尚未运行
startCountDown	void startCountDown(int totalSeconds)	totalSeconds：总秒数	无	totalSeconds > 0	重置累计秒数并开始倒计时
update	void update(double deltaSeconds)	deltaSeconds：本帧秒数	无	运行中且 deltaSeconds >= 0	累计满 1 秒则递减，归零时仅回调一次
stop	void stop()	无	无	无	running=false，保留剩余秒数

方法名	方法签名	参数说明	返回值	前置条件	后置条件
getSecondsCount	int getSecondsCount()	无	剩余秒数	无	不修改状态

2.6 其余模型与服务骨架

类别	主要类型	当前设计落点	实现状态
订单	Order Recipe OrderResult	订单字段、菜谱需求、结果字段	数据模型已定义；字段类型仍较粗，如 remainingTime 和 success 使用 String
食材	Ingredient IngredientType IngredientStatus	食材类型、RAW CUT COOKED 状态和进度	仅 FISH 一种类型，未接入游戏循环
工作台	Station StationType	食材库、切菜板、煎锅、煮锅、搅拌机、出餐口和垃圾桶枚举	模型存在，交互逻辑未实现
装盘	Plate	Set<Ingredient> contents	模型存在，未与订单提交集成
订单服务	OrderService OrderServiceImpl	创建、查询、更新和提交接口	实现方法为 null、List.of 或空方法
评分	ScoreService CalculateScore	成功分、小费、罚分接口；星级阈值计算	ScoreService 无实现；CalculateScore 未接入游戏
其他服务	GameService PlayerService KitchenService	分层接口占位	接口为空；仅 GameService 和 PlayerService 有空实现类
关卡配置	GameConfig GameMap	关卡与 13×9 地图尺寸占位	GameConfig 为空；GameMap 仅静态行列字段

三 P0 设计落点对照

P0 功能点来自仓库中的需求分析.md。设计落点按当前源码判断，不把未接入的枚举、接口或 README 描述记为完成。

P0 功能点	设计落点	状态	设计说明
暂停、重新开始、退出	StartController.onQuitButtonClick ; ResultController.restartGame; GameState.PAUSED	部分实现	退出与重启回调存在；暂停入口、暂停计时和恢复流程未实现
玩家全向移动、拿取物品、操作设备	GameView 键盘映射； GameController.press/release； Player.update	部分实现	WASD 移动和边界限制已实现；拿取与设备交互未实现
食材库无限拿取并区分种类	StationType.INGREDIENT_SOURCE； IngredientType	未实现	仅有类型占位；IngredientType 当前只有 FISH，无拿取服务
顾客限时持续提出订单	Order Recipe； OrderService.createRandomOrder/upd ateOrders	未实现	OrderServiceImpl 返回 null、空列表或空操作
完成订单后评分和时效小费	ScoreService；CalculateScore	未实现	评分接口与星级算法存在，但无 ScoreService 实现且未接入订单
初始页、关卡选择页、两个以上游戏场景	start.fxml game.fxml result.fxml； Launcher	部分实现	三类页面资源存在；关卡选择页和两个不同游戏地图未实现，result.fxml 为空
游戏计时器	GameTimer；GameView.renderTime	已实现	60 秒倒计时、mm:ss 展示、最后 10 秒红色提示和归零结束已接通
角色移动动画	AnimationTimer；PlayerView.render	部分实现	逐帧刷新已实现；角色目前仅为红色矩形，没有精灵帧动画
背景音乐和音效	Launcher.playMenuMusic； startmenu.mp3	部分实现	开始菜单音乐可循环播放；仓库未见游戏过程音效资源和调用
煎锅、煮锅、切菜台、搅拌机	StationType 枚举	未实现	类型名称已定义，设备实体、加工进度和交互服务未实现
出餐口	StationType.SERVING_COUNTER； OrderService.submitPlate	未实现	类型与接口存在，提交实现返回 null
盘子组合食材形成菜肴	Plate.contents； Recipe.requirements	未实现	数据结构存在，匹配、组合和转移规则未实现

P0 功能点	设计落点	状态	设计说明
垃圾桶丢弃食材和菜肴	StationType.TRASH_BIN	未实现	只有枚举值，无清空物品交互

3.1 设计模式使用说明

设计决策	使用位置	选择理由或当前限制
MVC 式分层	model controller view service 包	显示、状态协调、数据模型和服务接口分开；但 GameView 同时处理输入和绘制，属于轻量实现
回调注入	Launcher.configureController; GameController.onGameFinished	用 Runnable 解耦页面控制器与 Stage；没有引入额外事件总线
帧时间驱动	GameView.startGameLoop; Player.update; GameTimer.update	移动与计时使用 deltaSeconds，不依赖固定帧率；每帧时间上限为 0.05 秒
接口加多实现的策略模式	OrderService ScoreService 等	当前没有多个有效实现，不能认定已形成策略模式；接口多为骨架
观察者式变化通知	Canvas focusedProperty 监听器	仅对焦点变化使用 JavaFX 监听器；游戏状态与订单尚未建立统一观察者机制
未使用的抽象层	GameService PlayerService KitchenService 及空实现	目前没有行为或调用方；应在功能落地时补全，否则后续可删除。是否原计划删除待确认

四 AI 使用与核对说明

4.1 AI 使用记录

仓库历史提交 84cd710 中保存了 AI 记录.md，可验证以下使用记录。记录未逐条注明日期，也未说明每段示例是否直接复制进源码。

序号	AI 工具	生成内容	提示词摘要
1	Gemini 3.6 Flash	中英文项目名称建议、README 模板、倒计时示例	游戏命名；README 模板；JavaFX 正计时和倒计时设计
2	GPT 5.5	10 天 4 人开发路线和功能取舍	检查 README 并规划实训开发路线
3	GPT 5.6	JavaFX Canvas 使用、绘图和动画示例	Canvas 如何应用于 JavaFX 项目

序号	AI 工具	生成内容	提示词摘要
4	OpenAI Codex	本详细设计说明书；未生成项目代码	依据模板和仓库证据生成详细设计，并标注不确定项

4.2 AI 生成代码核对

4.2.1 倒计时设计

AI 记录中的原始示例把 Timeline、Label 和按钮事件写在同一个 Application 中，并由 updateTimer 直接更新 UI。

```
timeline = new Timeline(new KeyFrame(Duration.seconds(1), e -> updateTimer()));

private void updateTimer() {
    secondsCount--;
    if (secondsCount <= 0) {
        timeline.stop();
        onTimeUp();
    }
    renderTime();
}
```

问题类型	位置	问题描述	修正方式
职责耦合	AI 示例 GameTimerApp	计时状态、JavaFX 控件和按钮事件集中在 Application，难以独立测试或复用	当前 GameTimer 不依赖 JavaFX；GameView 负责显示，GameController 负责结束回调
与游戏循环重复	Timeline 每秒触发	项目已有 AnimationTimer；再维护 Timeline 会形成两个时间源	当前由 GameView 统一传入 deltaSeconds，GameTimer 累计满一秒再递减

当前实现的核心代码如下。它修正了职责耦合和双时间源问题，但没有 pause 或 resume 方法，因此 P0 暂停功能仍未完成。

```
public void update(double deltaSeconds) {
    if (!running) return;
    if (deltaSeconds < 0) {
        throw new IllegalArgumentException("Delta seconds cannot be negative.");
    }
    accumulatedSeconds += deltaSeconds;
    while (running && accumulatedSeconds >= 1) {
        accumulatedSeconds -= 1;
    }
}
```

```
secondsCount--;  
if (secondsCount <= 0) {  
    secondsCount = 0;  
    running = false;  
    onTimeUp.run();  
}  
}  
}
```

4.2.2 Canvas 移动设计

AI 记录中的 Canvas 动画示例按每帧固定增加 2 像素，并把绘制与位置更新写在同一段 handle 逻辑中。

```
public void handle(long now) {  
    gc.clearRect(0, 0, canvas.getWidth(), canvas.getHeight());  
    gc.fillOval(x, 100, 50, 50);  
    x += 2;  
}
```

问题类型	位置	问题描述	修正方式
帧率相关	AI 示例 <code>x += 2</code>	不同帧率会产生不同移动速度	Player.update 使用 <code>speed × deltaSeconds</code> ，默认速度 220 像素每秒
职责混合	AI 示例 <code>handle</code>	位置状态和绘制代码混在 <code>AnimationTimer</code> 内	当前 Player 保存状态，GameController 更新，PlayerView 绘制，GameView 驱动循环
时间突跳	页面卡顿后的首帧	过大的时间差可能让玩家或计时器一次跳过多	GameView 将单帧 <code>deltaSeconds</code> 限制为 0.05；该取舍会在卡顿时减慢游戏时间

当前实现将输入、状态更新和绘制分开，主要代码如下。

```
double deltaSeconds = lastTime == 0  
    ? 0  
    : Math.min((now - lastTime) / 1_000_000_000.0, 0.05);  
lastTime = now;  
controller.update(deltaSeconds);  
renderFrame(graphics);
```

4.3 AI 核对总结

项目	结论
AI 生成方法数	不确定。AI 历史记录包含多段完整示例，但未标记哪些方法被直接采用

项目	结论
本次可核对示例	2 组：倒计时设计、Canvas 移动设计
发现并已处理的问题	可确认 4 类改进：职责耦合、双时间源、帧率相关移动、绘制与状态混合；当前源码已分别通过分层与 deltaSeconds 处理
仍未解决的问题	暂停恢复、订单与烹饪闭环、结果页 UI、单元测试、对角线速度归一化、参数空值或负值校验
后续使用 AI 注意事项	保留提示词和原始输出；逐方法记录采用与修改情况；合入前编译并补充单元测试；不把示例目标写成已实现事实